
Source Code Listing

OncoScreen Disease Prediction

CodeAlpha Machine Learning Internship — Task 4 (2 of 2 tasks completed)

SUMMARY

- Full source of disease_prediction.py (training/evaluation) and CodeAlpha_DiseasePrediction.html (deployed tool).
- Companion document: CodeAlpha_DiseasePrediction_ResearchPaper.pdf.

Author: Mayank Swaraj (Mayur Krishna)
Repository naming: CodeAlpha_DiseasePrediction
Date: July 2026

Source File: disease_prediction.py

```

1 | #!/usr/bin/env python3
2 | """
3 | CodeAlpha ML Task 4 – Disease Prediction from Medical Data
4 | Predicts malignant vs. benign breast tumors using the real Wisconsin
5 | Diagnostic Breast Cancer dataset, comparing a full-feature Random Forest
6 | benchmark against a distilled, transparent 8-feature Logistic Regression
7 | deployed for browser inference.
8 | """
9 |
10 | import numpy as np
11 | from sklearn.datasets import load_breast_cancer
12 | from sklearn.model_selection import train_test_split
13 | from sklearn.ensemble import RandomForestClassifier
14 | from sklearn.linear_model import LogisticRegression
15 | from sklearn.preprocessing import StandardScaler
16 | from sklearn.metrics import roc_auc_score, accuracy_score, precision_score, recall_score, f1_score
17 |
18 |
19 | def evaluate(y_true, y_pred, y_proba, label):
20 |     print(f"\n--- {label} ---")
21 |     print(f"AUC:         {roc_auc_score(y_true, y_proba):.4f}")
22 |     print(f"Accuracy:    {accuracy_score(y_true, y_pred):.4f}")
23 |     print(f"Precision:  {precision_score(y_true, y_pred):.4f}")
24 |     print(f"Recall:     {recall_score(y_true, y_pred):.4f}")
25 |     print(f"F1:        {f1_score(y_true, y_pred):.4f}")
26 |
27 |
28 | def main():
29 |     data = load_breast_cancer()
30 |     X, y = data.data, data.target
31 |     feat_names = list(data.feature_names)
32 |     print("Samples:", X.shape, "| classes:", data.target_names)
33 |
34 |     Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
35 |
36 |     # Primary benchmark: Random Forest on all 30 features
37 |     rf = RandomForestClassifier(n_estimators=200, max_depth=6, random_state=42)
38 |     rf.fit(Xtr, ytr)
39 |     proba_rf = rf.predict_proba(Xte)[: , 1]
40 |     evaluate(yte, rf.predict(Xte), proba_rf, "Random Forest (30 features)")
41 |
42 |     importances = rf.feature_importances_
43 |     top_idx = np.argsort(importances)[::-1][:8]
44 |     top_features = [feat_names[i] for i in top_idx]
45 |     print("\nTop 8 features by importance:", top_features)
46 |
47 |     # Deployed model: Logistic Regression on the top-8 features (transparent, browser-sized)
48 |     Xsub = X[:, top_idx]
49 |     Xtr2, Xte2, ytr2, yte2 = train_test_split(Xsub, y, test_size=0.2, random_state=42, stratify=y)
50 |     scaler = StandardScaler()
51 |     Xtr2_s, Xte2_s = scaler.fit_transform(Xtr2), scaler.transform(Xte2)
52 |     lr = LogisticRegression(max_iter=2000)
53 |     lr.fit(Xtr2_s, ytr2)
54 |     proba_lr = lr.predict_proba(Xte2_s)[: , 1]
55 |     evaluate(yte2, lr.predict(Xte2_s), proba_lr, "Deployed Logistic Regression (8 features)")
56 |
57 |
58 | if __name__ == "__main__":
59 |     main()
60 |

```

Source File: CodeAlpha_DiseasePrediction.html

```

1 | <!DOCTYPE html>
2 | <html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,
3 |   initial-scale=1.0">
4 | <title>OncoScreen – Disease Prediction</title>
5 | <style>
6 |   :root{--bg:#F4F7F8;--panel:#FFFFFF;--border:#E1E8EB;--teal:#0D7A76;--teal-dark:#095652;--red:#C0
7 |     392B;--amber:#C77F1A;--text:#20282A;--muted:#77878C;}
8 |   *{box-sizing:border-box;}html,body{margin:0;padding:0;background:var(--bg);}
9 |   body{font-family:'Inter',system-ui,-apple-system,sans-serif;color:var(--text);display:flex;justi

```

```

10 |     fy-content:center;}}
11 | #app{width:100%;max-width:480px;min-height:100vh;background:var(--bg);}
12 | header{padding:20px 18px 14px;background:var(--panel);border-bottom:1px solid var(--border);}
13 | .brand{font-family:'Space
14 |     Grotesk','Inter',sans-serif;font-weight:700;font-size:19px;color:var(--teal-dark);}
15 | .brand span{color:var(--teal);}
16 | .sub{font-size:11px;color:var(--muted);margin-top:4px;}
17 | #metrics{display:grid;grid-template-columns:repeat(4,1fr);gap:1px;background:var(--border);border-
18 |     bottom:1px solid var(--border);}
19 | .metric{background:var(--panel);padding:10px 6px;text-align:center;}
20 | .metric .val{font-family:'JetBrains
21 |     Mono',monospace;font-size:14px;font-weight:700;color:var(--teal-dark);}
22 | .metric .lbl{font-size:8.5px;color:var(--muted);text-transform:uppercase;margin-top:2px;}
23 | #form{padding:16px 18px 8px;}
24 | .field{margin-bottom:16px;}
25 | .field-head{display:flex;justify-content:space-between;font-size:12.5px;margin-bottom:4px;}
26 | .field-name{font-weight:600;color:var(--text);}
27 | .field-val{font-family:'JetBrains Mono',monospace;color:var(--teal-dark);font-weight:700;}
28 | input[type=range]{width:100%;height:5px;border-radius:3px;background:var(--border);appearance:none;outline:none;}
29 | input[type=range]::-webkit-slider-thumb{appearance:none;width:18px;height:18px;border-radius:50%;background:var(--teal);cursor:pointer;border:2px solid white;box-shadow:0 0 0 1px
30 |     var(--border);}
31 | .field-range{display:flex;justify-content:space-between;font-size:9px;color:var(--muted);margin-
32 |     top:2px;}
33 | #resultCard{margin:8px 18px
34 |     16px;padding:16px;border-radius:14px;background:var(--panel);border:1px solid var(--border);}
35 | #resultCard.malignant{border-color:var(--red);}#resultCard.benign{border-color:var(--teal);}
36 | .result-label{font-size:11px;color:var(--muted);text-transform:uppercase;letter-spacing:.04em;margin-bottom:6px;}
37 | .result-main{display:flex;align-items:baseline;gap:8px;}
38 | .result-class{font-family:'Space Grotesk','Inter',sans-serif;font-weight:700;font-size:20px;}
39 | .result-class.malignant{color:var(--red);}#result-class.benign{color:var(--teal-dark);}
40 | .result-prob{font-family:'JetBrains Mono',monospace;font-size:13px;color:var(--muted);}
41 | .gauge{height:8px;border-radius:4px;background:var(--border);margin-top:12px;overflow:hidden;}
42 | .gauge-fill{height:100%;border-radius:4px;transition:width .3s;}
43 | .disclaimer{font-size:10px;color:var(--muted);margin-top:12px;line-height:1.5;padding-top:10px;border-top:1px solid var(--border);}
44 | #presets{padding:0 18px 24px;display:flex;gap:8px;}
45 | .preset-btn{flex:1;font-size:11px;font-weight:600;padding:9px 6px;border-radius:8px;border:1px solid var(--border);background:var(--panel);color:var(--text);cursor:pointer;}
46 | </style></head>
47 | <body><div id="app">
48 |   <header><div class="brand">Onco<span>Screen</span></div>
49 |   <div class="sub">CodeAlpha ML Task 4 – Disease Prediction (real Wisconsin Breast Cancer
50 |     dataset)</div></header>
51 |   <div id="metrics">
52 |     <div class="metric"><div class="val" id="mPrec">--</div><div class="lbl">Precision</div></div>
53 |     <div class="metric"><div class="val" id="mRec">--</div><div class="lbl">Recall</div></div>
54 |     <div class="metric"><div class="val" id="mF1">--</div><div class="lbl">F1-Score</div></div>
55 |     <div class="metric"><div class="val" id="mAuc">--</div><div class="lbl">ROC-AUC</div></div>
56 |   </div>
57 |   <div id="presets">
58 |     <button class="preset-btn" id="presetBenign">Load benign-like sample</button>
59 |     <button class="preset-btn" id="presetMalignant">Load malignant-like sample</button>
60 |   </div>
61 |   <div id="form"></div>
62 |   <div id="resultCard">
63 |     <div class="result-label">Prediction</div>
64 |     <div class="result-main"><span class="result-class" id="resultClass">--</span><span
65 |       class="result-prob" id="resultProb"></span></div>
66 |     <div class="gauge"><div class="gauge-fill" id="gaugeFill"></div></div>
67 |     <div class="disclaimer">Trained on the Wisconsin Diagnostic Breast Cancer dataset (569
68 |       samples).
69 |       Educational internship deliverable, not a diagnostic device.</div>
70 |   </div>
71 | </div>
72 | <script>
73 | const MODEL = {"features":["worst perimeter","worst area","worst concave points","mean concave
74 |   points","worst radius","mean radius","mean perimeter","mean area"],"scaler_mean":[106.625297,869.0
75 |   26593,0.113904,0.048344,16.177226,14.067213,91.557407,648.541099],"scaler_scale":[33.158555,552.31
76 |   8965,0.066711,0.038882,4.764775,3.495532,24.122679,344.565295],"coef":[-1.260692,-1.819295,-1.7139
77 |   49,-1.021987,-1.678933,0.360331,0.383447,0.002115],"intercept":0.485934,"ranges":{"worst
78 |   perimeter":[50.41,251.2],"worst area":[185.2,4254.0],"worst concave points":[0.0,0.291],"mean
79 |   concave points":[0.0,0.201],"worst radius":[7.93,36.04],"mean radius":[6.981,28.11],"mean
80 |   perimeter":[43.79,188.5],"mean area":[143.5,2501.0]},"metrics":{"rf_full":{"auc":0.994,"accuracy":
81 |   0.9474,"precision":0.9583,"recall":0.9583,"f1":0.9583},"deployed":{"auc":0.9927,"accuracy":0.9474,
82 |   "precision":0.9714,"recall":0.9444,"f1":0.9577}}};

```

```

88 | const RANGES = {};
89 | MODEL.features.forEach(f => { RANGES[f] = MODEL.ranges[f]; });
90 | const BENIGN_SAMPLE = {}, MALIGNANT_SAMPLE = {};
91 | MODEL.features.forEach((f,i)=>{
92 |   const [lo,hi] = RANGES[f];
93 |   BENIGN_SAMPLE[f] = +(lo + (hi-lo)*0.2).toFixed(3);
94 |   MALIGNANT_SAMPLE[f] = +(lo + (hi-lo)*0.75).toFixed(3);
95 | });
96 | const formEl = document.getElementById('form');
97 | const sliders = {};
98 | function cssId(name){ return name.replace(/\s+/g, '_'); }
99 | MODEL.features.forEach(name=>{
100 |   const [min,max] = RANGES[name];
101 |   const def = +((min+max)/2).toFixed(3);
102 |   const field = document.createElement('div');
103 |   field.className='field';
104 |   field.innerHTML = `<div class="field-head"><span class="field-name">${name}</span><span
105 |     class="field-val" id="val_${cssId(name)}">${def}</span></div>
106 |     <input type="range" id="slider_${cssId(name)}" min="${min}" max="${max}"
107 |       step="${(max-min)/200}" value="${def}">
108 |     <div class="field-range"><span>${min}</span><span>${max}</span></div>`;
109 |   formEl.appendChild(field);
110 |   const slider = field.querySelector('input');
111 |   sliders[name]=slider;
112 |   slider.addEventListener('input', ()=>{
113 |     document.getElementById(`val_${cssId(name)}`).textContent =
114 |       parseFloat(slider.value).toFixed(name.includes('concave')?3:1);
115 |     predict();
116 |   });
117 | });
118 | function predict(){
119 |   let z = MODEL.intercept;
120 |   MODEL.features.forEach((name,i)=>{
121 |     const raw = parseFloat(sliders[name].value);
122 |     const std = (raw - MODEL.scaler_mean[i]) / MODEL.scaler_scale[i];
123 |     z += MODEL.coef[i]*std;
124 |   });
125 |   const probBenign = 1/(1+Math.exp(-z));
126 |   const probMalignant = 1-probBenign;
127 |   const isMalignant = probMalignant>0.5;
128 |   const card = document.getElementById('resultCard');
129 |   const cls = document.getElementById('resultClass');
130 |   const prob = document.getElementById('resultProb');
131 |   const fill = document.getElementById('gaugeFill');
132 |   card.className = isMalignant?'malignant':'benign';
133 |   cls.className = 'result-class '+isMalignant?'malignant':'benign';
134 |   cls.textContent = isMalignant?'Malignant risk':'Benign pattern';
135 |   prob.textContent = `${(probMalignant*100).toFixed(1)}% malignant probability`;
136 |   fill.style.width = `${(probMalignant*100).toFixed(1)}%`;
137 |   fill.style.background = isMalignant?'var(--red)':'var(--teal)';
138 | }
139 | function loadSample(sample){
140 |   MODEL.features.forEach(name=>{
141 |     sliders[name].value = sample[name];
142 |     document.getElementById(`val_${cssId(name)}`).textContent =
143 |       parseFloat(sample[name]).toFixed(name.includes('concave')?3:1);
144 |   });
145 |   predict();
146 | }
147 | document.getElementById('presetBenign').addEventListener('click', ()=>loadSample(BENIGN_SAMPLE));
148 | document.getElementById('presetMalignant').addEventListener('click',
149 |   ()=>loadSample(MALIGNANT_SAMPLE));
150 | document.getElementById('mPrec').textContent =
151 |   (MODEL.metrics.deployed.precision*100).toFixed(1)+'%';
152 | document.getElementById('mRec').textContent = (MODEL.metrics.deployed.recall*100).toFixed(1)+'%';
153 | document.getElementById('mF1').textContent = (MODEL.metrics.deployed.f1*100).toFixed(1)+'%';
154 | document.getElementById('mAuc').textContent = MODEL.metrics.deployed.auc.toFixed(3);
155 | predict();
156 | </script></body></html>

```